Black-box error reduction for weighted PRGs against read-once branching programs

Gil Cohen, Dean Doron, Oren Renard,
Ori Sberlo, Amnon Ta-Shma

doi:10.4230/LIPIcs.CCC.2021.22; CC BY 4.0

Editorial scope. Counted once: complete black-box weighted-PRG error reduction, including its seed/space/accuracy parameters and full underlying local proofs. Nisan substitution and direct corollaries remain context. Arbitrary-alphabet intermediate programs, signed unbounded weights, matrix norm and epsilon parameter roles preserved; no author mathematical repair or supplementary proof.

Allowed standard prerequisites (editorial).

- Induced matrix norms, read-once branching programs, standard Nisan/INW generators and space composition. Exact Definitions1/4/5/6 and imported Theorems7/8/9 retained. Full source-local Richardson proof and all Nisan/high-entropy-extractor/INW space implementation appendices included, including original Proof sketch labels. All published bibliography entries retained; independent imported background not recursively reproved.

Author main result

Theorem2 (main result, see also Corollary15). Suppose PRG is an (n, w, n^{-2}) PRG with seed length s_0, computable in space m. Then, for every ϵ there exists an (n, w, ϵ) WPRG with seed length

$$s = s_0 + O\left(\log \frac{w}{\epsilon} \cdot \log\log_n \frac{1}{\epsilon}\right)$$

that is computable in space $O(m + (\log\log(w/\epsilon))^3)$.

Necessary ROBP and weighted-distribution context (Sections1.2–1.3)

A length n, width w read-once branching program (ROBP) is a directed graph whose nodes, called states, are partitioned to $n + 1$ layers, each consists of at

most w states. The first layer contains a designated "start" state, and the last layer consists of two states labeled 'accept' and 'reject'. From every state but for the latter two, there are two outgoing edges, labeled by 0 and 1, to the following layer.[1] On input $x \in \{0,1\}^n$, the computation proceeds by following the edges according to the labels given by the bits of x starting from the start state. The string x is accepted by the program if the computation ends in the accept state. We say that a distribution D on n-bit strings is (n, w, ϵ) pseudorandom if for every length n, width w ROBP, the path induced by an instruction sequence that is sampled from D has, up to an additive error ϵ, the same probability to end in the accept state as a truly random path. A truly random path corresponds to a path picked uniformly at random from the 2^n possible paths. An (n, w, ϵ) pseudorandom generator (PRG) is an algorithm $\mathrm{PRG} : \{0,1\}^s \to \{0,1\}^n$ that when fed with s uniformly random bits has an output distribution that is (n, w, ϵ) pseudorandom. We refer to the input to PRG as the seed.

Definition1 (pseudorandom pseudo-distribution). Let $\rho_1, \ldots, \rho_{2^s} \in \mathbb{R}$ and $p_1, \ldots, p_{2^s} \in \{0,1\}^n$. The sequence $\widetilde{D} = ((\rho_1, p_1), \ldots, (\rho_{2^s}, p_{2^s}))$ is an (n, w, ϵ) pseudorandom pseudo-distribution (PRPD) if for every length n, width w ROBP, the sum of all ρ_i-s for which the respective paths p_i end in the accept state is an ϵ-approximation to the probability of ending in the accept state by taking a truly random path in the program.

Note that Definition1 allows the weights ρ_i to take both positive and negative values. These values are not necessarily bounded by 1 in absolute value, nor by any constant for that matter, and they do not necessarily sum up to 1. Nevertheless, the definition requires that the numbers cancel out nicely so that summing the weights of the respective paths that arrive to the accept state yields an ϵ-approximation for the probability of arriving to the accept state by taking a truly random path (and, in particular, the sum is a number in $[-\epsilon, 1+\epsilon]$). Analogous to a PRG, an (n, w, ϵ) weighted pseudorandom generator (WRPG) is an algorithm $\mathrm{WPRG} : \{0,1\}^s \to \mathbb{R} \times \{0,1\}^n$ whose output, when fed with a uniform seed, is an (n, w, ϵ) PRPD. A WPRG that can be computed in bounded space suffices to derandomize two-sided error randomized algorithms. Indeed, the straightforward derandomization using a pseudorandom (proper) distribution, which sums the probability mass of the relevant paths, works just as well for pseudo-distributions as one can sum up the weights ρ_i which, in a sense, generalize the probability mass. Of course, the space requirement now depends on the bit complexity of the weights as well.

2 Preliminaries

2.1 Matrices, branching programs, and space complexity

A matrix is Boolean if all its entries are in $\{0,1\}$, and stochastic if all its entries are nonnegative and the sum of each column is 1. Denote by $\mathrm{BSto}(w)$ the set of $w \times w$ boolean stochastic matrices. We will denote by $\|\cdot\|$ the induced ℓ_1

[1] For simplicity, here we only consider ROBPs with two outgoing edges. Larger out-degrees (or alphabet) can also be considered and is in fact crucial for obtaining our result even if one is only interested in the binary case.

norm, i.e., $\|A\| = \max_j \sum_i |A_{i,j}|$. We will often work with block matrices. For instance, we may interpret $A \in \mathbb{R}^{nm \times nm}$ as an $n \times n$ matrix with entries which are $m \times m$ matrices. Whenever this interpretation is clear, we let $A[i,j]$ be the (i,j)-th block. In this example, $A[i,j] \in \mathbb{R}^{m \times m}$.

Definition4 (branching program). Let Σ be some alphabet and let $n, w \in \mathbb{N}$. An (n, Σ, w) branching program (BP) is a sequence $B = (B_1, \ldots, B_n)$, where each $B_i : \Sigma \to \mathrm{BSto}(w)$.

For $b \leq a$ we let $B_{[b,a]}$ be the $(a - b + 1, \Sigma, w)$ BP $(B_a, \ldots, B_b)$.

Definition5. The value of an (n, Σ, w) BP $B = (B_1, \ldots, B_n)$ on $x = (x_1, \ldots, x_n) \in \Sigma^n$, denoted $\mathrm{val}(B, x)$, is the realized $w \times w$ matrix of B when fed by x, i.e.

$$\mathrm{val}(B, x) = B_n(x_n) \cdot B_{n-1}(x_{n-1}) \cdots B_1(x_1).$$

If B is the empty sequence, we set $\mathrm{val}(\emptyset, x) = I_w$.

Definition6 (weighted PRG). We say W is an (n, Σ, w, ϵ)-WPRG against BPs with seed length s if:

- $W = (I, \mu)$ where $I : \{0,1\}^s \to \Sigma^n$ and $\mu : \{0,1\}^s \to \mathbb{R}$, and,
- For every (n, Σ, w) BP $B = (B_1, \ldots, B_n)$, it holds that

$$\left| \mathbb{E}_{x \in \{0,1\}^s}[\mu(x) \cdot \mathrm{val}(B, I(x))] - \mathbb{E}_{x \in \Sigma^n}[\mathrm{val}(B, x)] \right| \leq \epsilon.$$

When $\mu \equiv 1$, we say that W is a PRG. For $1 \leq \ell \leq n$ we let $G_\ell : \{0,1\}^{s_0} \to \Sigma^\ell$ be the first ℓ symbols of the output of G. Note that if $G : \{0,1\}^{s_0} \to \Sigma^n$ is an (n, Σ, w, ϵ) PRG then G_ℓ is an $(\ell, \Sigma, w, \epsilon)$ PRG. We say $f : \Lambda_1 \to \Lambda_2$ is computable in space s, if given $x \in \Lambda_1$ and index j, $f(x)_j \in \Lambda_2$ can be computed in additional work space that consists of s bits. We will use the following well known theorem regarding the space complexity of compositions.

Theorem7. Let $f_1, f_2 : \{0,1\}^* \to \{0,1\}^*$ be two functions that can be computed in $s_1, s_2 : \mathbb{N} \to \mathbb{N}$ space such that $s_1(n), s_2(n) = \Omega(\log n)$. Then, on input x, $f_2 \circ f_1 : \{0,1\}^* \to \{0,1\}^*$ can be computed using $O(s_1(|x|) + s_2(|f_1(x)|))$ space.

2.2 Known PRG constructions

Theorem8 ([17,18]). For any positive integers n, w, any error parameter $\epsilon > 0$ and any alphabet Σ, there exists an (n, Σ, w, ϵ) PRG with seed length

$$s = O\left(\log n \cdot \log \frac{nw|\Sigma|}{\epsilon}\right),$$

computable in space $\min\{O(\log(nw|\Sigma|/\epsilon)), O(\log n \cdot \log\log(nw|\Sigma|/\epsilon))\}$.

Theorem9 ([15]). For any positive integers n, w, any error parameter $\epsilon > 0$ and any alphabet Σ, there exists an (n, Σ, w, ϵ) PRG with seed length

$$s = \log|\Sigma| + O(\log n \cdot \log(nw/\epsilon)),$$

computable in space $O(\log n \cdot (\log\log(nw|\Sigma|/\epsilon))^2)$.

Theorem8 is derived almost directly from [17,18], and Theorem9 follows from [15], except for the space complexity which is implicit in those works and also depends on the specific implementation. For completeness, we give the proof of Theorem8 in AppendixB.1, and of Theorem9 in AppendixB.3.

3 Richardson iteration

Let A be an invertible $n \times n$ real matrix, and assume that B approximates A^{-1}, concretely, $\|B - A^{-1}\| \leq \epsilon_0$ for some sub-multiplicative norm. Richardson iteration is a method for obtaining a more refined approximation of A^{-1} given access to the crude B as well as to the original matrix A.

Lemma10. Let $L \in \mathbb{R}^{m\times m}$ be an invertible matrix and $A \in \mathbb{R}^{m\times m}$ such that $\|L^{-1} - A\| \leq \epsilon_0$. For any nonnegative integer k, define

$$R(A, L, k) = \sum_{i=0}^{k} (I - AL)^i A.$$

Then, $\|L^{-1} - R(A, L, k)\| \leq \|L^{-1}\| \cdot \|L\|^{k+1} \cdot \epsilon_0^{k+1}$.

The proof is deferred to AppendixA. Following [1] we will be interested in the following instantiation of the Richardson iteration. Let $\mathbf{M} = (M_1, \ldots, M_n)$ be a sequence of $w \times w$ matrices. We consider the $(n+1)w \times (n+1)w$ matrix

$$M = \begin{pmatrix} 0 & 0 & \cdots & 0 & 0 \\ M_1 & 0 & \cdots & 0 & 0 \\ 0 & M_2 & \ddots & 0 & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & \cdots & M_n & 0 \end{pmatrix}. \tag{4}$$

The Laplacian of $\mathbf{M}$ is $L = I_{(n+1)w} - M$, and we treat L as an $(n+1) \times (n+1)$ block matrix. The following claim follows by a simple calculation.

Claim11. For $i, j \in [n+1]$, the (i, j)-th block of L^{-1} is given by

$$L^{-1}[i, j] = \begin{cases} M_{i-1} \cdots M_j & i > j, \\ I_w & i = j, \\ 0 & i < j. \end{cases}$$

Richardson for branching programs. Let $B = (B_1, \ldots, B_n)$ be an (n, Σ, w) BP and let $M_i = \mathbb{E}_{\sigma\in\Sigma}[B_i(\sigma)]$ be the corresponding transition matrices. Thus, approximating the transition probabilities of B,

$$\mathbb{E}_{x\in\Sigma^n}[\mathrm{val}(B, x)] = M_n \cdots M_1,$$

amounts to approximating the lowest leftmost entry $L^{-1}[n+1, 1]$.

Claim12. Let $B = (B_1, \ldots, B_n)$ be an (n, Σ, w) BP. Set $M_i = \mathbb{E}_{\sigma \in \Sigma}[B_i(\sigma)]$ and L as in Equation(4). Also, let $G : \{0,1\}^s \to \Sigma^n$ be an $(n, \Sigma, w, \epsilon_0)$ PRG and consider

$$A[a,b] = \begin{cases} \mathbb{E}_{x \in \{0,1\}^s}[\mathrm{val}(B_{[b,a-1]}, G_{a-b}(x))], & a \geq b, \\ 0 & a < b. \end{cases} \tag{5}$$

Then, $\|L^{-1} - R(A, L, k)\| \leq (n+1) \cdot (2\epsilon_0)^{k+1}$.

Let A as in Equation(5) and write $R(A, L, k) = \sum_{i=0}^{k} \Delta^i A$ where $\Delta = I - AL$. Denote $A' = A - I$, i.e., A' is the part of A below the main diagonal. Then,

$$\Delta = I - AL = I - A(I - M) = (I - A) + AM = AM - A'.$$

In block notation, for $a, b \in [n+1]$, following Equation(4),

$$AM[a,b] = \sum_{i=1}^{n+1} A[a,i]M[i,b] = A[a,b+1]M[b+1,b] = A[a,b+1] \cdot M_b.$$

Thus,

$$\Delta[a,b] = \begin{cases} A[a,b+1] \cdot M_b - A[a,b] & a > b, \\ 0 & a \leq b. \end{cases} \tag{6}$$

Going back to $R(A, L, k)$, for $a > b$ we have that

$$R(A,L,k)[a,b] = A[a,b] + \sum_{i=1}^{k} \sum_{a > r_i > \cdots > r_1 \geq b} \Delta[a, r_i] \cdot \Delta[r_i, r_{i-1}] \cdots \Delta[r_2, r_1] \cdot A[r_1, b]. \tag{7}$$

If we further let $C_0[a,b] = A[a,b+1] \cdot M_b$ and $C_1[a,b] = A[a,b]$, then

$$R(A,L,k)[a,b] = A[a,b] + \sum_{\substack{t \in \{0,1\}^i \\ a > r_i > \cdots > r_1 \geq b}} \sum_{t_1, \ldots, t_i \in \{0,1\}} (-1)^{t_1 + \cdots + t_i} \cdot C_{t_i}[a, r_i] \cdots C_{t_1}[r_2, r_1] \cdot A[r_1, b]. \tag{8}$$

4 The construction

4.1 Black-box error reduction

Let $G : \{0,1\}^{s_0} \to \Sigma^n$ be an $(n, \Sigma, w, \epsilon_G)$ and $G_{\mathrm{aux}} : \{0,1\}^{s_{\mathrm{aux}}} \to (\{0,1\}^{s_0} \times \Sigma)^{k+1}$ be a $(k+1, \{0,1\}^{s_0} \times \Sigma, w, \epsilon_{\mathrm{aux}})$ PRG. Also, for $t \in \{0,1\}$ and $\sigma \in \Sigma$ we let

$$G_{t,\ell}(x, \sigma) = \begin{cases} \sigma \circ G_{\ell-1}(x) & t = 0, \\ G_\ell(x) & t = 1. \end{cases} \tag{9}$$

We now define the WPRG $(I, \mu) : \{0,1\}^s \to \Sigma \times \mathbb{R}$. The seed $x \in \{0,1\}^s$ to our WPRG is interpreted as follows.

- The first $\log(k+1)$ bits encode $i \in \{0, \ldots, k\}$.
- The next $\log \binom{n}{i}$ bits encode a sequence $\ell = (\ell_0, \ell_1, \ldots, \ell_i)$ such that $\ell_0 + \cdots + \ell_i = n$, $\ell_i, \ldots, \ell_1 > 0$, and $\ell_0 \geq 0$.

- The next i bits are denoted by $t = t = (t_1, \ldots, t_i) \in \{0,1\}^i$.
- The next s_{aux} bits are denoted by $x_{\text{aux}} \in \{0,1\}^{s_{\text{aux}}}$.

Overall, we can write $x = (i, \ell, t, x_{\text{aux}})$, and the WPRG (I, μ) has seed length

$$s = s_{\text{aux}} + O(k \log n). \tag{10}$$

For brevity we sometimes omit the dependence of i, $(\ell_0, \ldots, \ell_i)$, $(t_1, \ldots, t_i)$, and x_{aux} on x. We define I and μ as follows.

$$I(x) = \begin{cases} G_n(G_{\text{aux}}(x_{\text{aux}})_0) & i = 0, \\ G_{t_i,\ell_i}(G_{\text{aux}}(x_{\text{aux}})_i) \circ \cdots \circ G_{t_1,\ell_1}(G_{\text{aux}}(x_{\text{aux}})_1) \circ G_{\ell_0}(G_{\text{aux}}(x_{\text{aux}})_0) & \text{otherwise.} \end{cases}$$

$$\mu(x) = \begin{cases} k+1 & i = 0, \\ (k+1) \cdot \binom{n}{i} \cdot 2^i \cdot (-1)^{t_1+\cdots+t_i} & \text{otherwise.} \end{cases} \tag{11}$$

where $G_{\text{aux}}(x_{\text{aux}})_j$ denotes the j'th symbol in $G_{\text{aux}}(x_{\text{aux}}) \in (\{0,1\}^{s_0} \times \Sigma)^{k+1}$. The weights are chosen so that the approximation yielded by the above WPRG is a derandomized version of Equation(8) for $(a, b) = (n+1, 1)$. Note that in Equation(8) we used $r_1, \ldots, r_i$ which partitioned the interval $[n+1, 1]$, while in Equation(11) we used $\ell_0, \ldots, \ell_i$ that sum to n. This is merely an alternative way of writing the sum – the ℓ_i-s are the sum of differences of the r_i-s.

4.2 Correctness

In this section we use the same notation as in Section3.

Lemma13. Let $0 < \epsilon < \epsilon_0 = 1/(4n)$ and let $k = \log_{1/\epsilon_0}(1/\epsilon)$. Suppose

- $G : \{0,1\}^{s_0} \to \Sigma^n$ is an $(n, \Sigma, w, \epsilon_G = \epsilon_0/(2(n+1)))$ PRG, and,
- $G_{\text{aux}} : \{0,1\}^{s_{\text{aux}}} \to (\{0,1\}^{s_0} \times \Sigma)^{k+1}$ is a $(k+1, \{0,1\}^{s_0} \times \Sigma, w, \epsilon_{\text{aux}} = \epsilon^3)$ PRG.

Then, (I, μ) is an (n, Σ, w, ϵ) WPRG with seed length $s = s_{\text{aux}} + O(\log(1/\epsilon))$ computable in space $O(\text{space}(G_{\text{aux}}) + \text{space}(G) + \log s)$.

Proof. Assume k, G and G_{aux} are as in the hypothesis of the lemma. The space complexity follows from Theorem7 and the seed length was analyzed in Equation(10). We are left to prove that (I, μ) is an (n, Σ, w, ϵ) WPRG. Fix any (n, Σ, w) BP $B = (B_1, \ldots, B_n)$. Let A be the $(n+1)w \times (n+1)w$ lower triangular block matrix in which

$$A[a, b] = \mathbb{E}_{x \in \{0,1\}^{s_0}}[\text{val}(B_{[b,a-1]}, G_{a-b}(x))]$$

for $a > b$, and $A[a, a] = I_w$. Since G is $(n, \Sigma, w, \epsilon_G = \epsilon_0/(2(n+1)))$ PRG we have that $\|L^{-1}[a, b] - A[a, b]\| \leq \epsilon_G$ and $\|L^{-1} - A\| \leq (n+1)\epsilon_G$. By our choice of μ,

$$\mathbb{E}_{x \in \{0,1\}^s}[\mu(x) \cdot \text{val}(B, I(x))] = \sum_{i=0}^{k} \sum_{t,\ell} (-1)^{t_1+\cdots+t_i} \cdot \mathbb{E}_{x_{\text{aux}}}[\text{val}(B, I(i, \ell, t, x_{\text{aux}}))],$$

and

$$R(A,L,k)[n+1,1] = A[n+1,1] + \sum_{i=1}^{k} \sum_{t,r} (-1)^{t_1+\cdots+t_i} \cdot C_{t_i}[n+1,r_i] \cdots C_{t_1}[r_2,r_1] \cdot A[r_1,1],$$

where $\ell_0 + \cdots + \ell_i = n$ and $n+1 > r_i > \cdots > r_1 \geq 1$. We soon prove:

Claim14. For every fixed $i \in \{0,\ldots,k\}$, $t \in \{0,1\}^i$, and ℓ such that $\ell_0 + \cdots + \ell_i = n$

$$\|\mathbb{E}_{x_{\text{aux}}}[\text{val}(B, I(i,\ell,t,x_{\text{aux}}))] - C_{t_i}[n+1,r_i] \cdots C_{t_1}[r_2,r_1] \cdot A[r_1,1]\| \leq \epsilon_{\text{aux}},$$

where $r_j = 1 + \ell_0 + \cdots + \ell_{j-1}$.

As we have at most $(k+1)n^k 2^k$ summands, we see that

$$\begin{aligned}\left\|\mathbb{E}_{x\in\{0,1\}^s}[\mu(x) \cdot \text{val}(B, I(x))] - R(A,L,k)[n+1,1]\right\| &\leq (k+1)n^k 2^k \cdot \epsilon_{\text{aux}} \\ &\leq \frac{n^{2k}}{2} \cdot \epsilon_{\text{aux}} \leq \frac{\epsilon}{2}.\end{aligned}$$

It therefore follows from Claim12 that

$$\begin{aligned}\|R(A,L,k)[n+1,1] - \mathbb{E}_{x\in\Sigma^n}[\text{val}(B,x)]\| &\leq (n+1)(2(n+1)\epsilon_G)^{k+1} \\ &\leq 2n \cdot \epsilon_0^{k+1} \leq 2n\epsilon_0\epsilon = \frac{\epsilon}{2},\end{aligned}$$

which together completes the proof. □

Proof of Claim14. Fix $i \in \{0,\ldots,k\}$, $\ell_0 + \cdots + \ell_i = n$, and $t \in \{0,1\}^i$ and recall that $r_j = 1 + \ell_0 + \cdots + \ell_{j-1}$. We define a $(k+1, \{0,1\}^{s_0} \times \Sigma, w)$ BP $B' = (B'_0, \ldots, B'_k)$ (that depends on i, ℓ, and t) such that for all $j = 0,\ldots,k$,

$$B'_j(x,\sigma) = \begin{cases} \text{val}(B_{[r_j, r_{j+1}-1]}, \sigma \circ G_{\ell_j - 1}(x)) & j > 0, t = 0, \\ \text{val}(B_{[r_j, r_{j+1}-1]}, G_{\ell_j}(x)) & j > 0, t = 1, \\ \text{val}(B_{[1, r_1 - 1]}, G_{\ell_0}(x)) & j = 0. \end{cases} \tag{12}$$

We stress that B'_j is a BP because a product of Boolean stochastic matrices is Boolean stochastic. The claim now follows since G_{aux} is a $(k+1, \{0,1\}^{s_0} \times \Sigma, w, \epsilon_{\text{aux}})$ PRG. □

4.3 The final construction

We now instantiate Lemma13 with G_{aux} being the INW PRG from Theorem9 and G being an arbitrary PRG. The reason for using the INW generator is its additive dependence on $\log|\Sigma|$.

Corollary15. Let $G : \{0,1\}^{s_0} \to \Sigma^n$ be an $(n, \Sigma, w, \epsilon_G)$. Then, for any error parameter $1/(4n) > \epsilon > 0$ there exists an (n, Σ, w, ϵ) WPRG with seed length

$$s_0 + O\left(\log\frac{w}{\epsilon} \cdot \log\log_n \frac{1}{\epsilon}\right)$$

computable in space $O(\text{space}(G) + \log\log_n(1/\epsilon) \cdot (\log\log(w/\epsilon))^2)$.

Had we used Nisan's PRG from Theorem8 instead of INW then the seed length would deteriorate to

$$O\left(s_0 \cdot \log\log_n \frac{1}{\epsilon} + \log \frac{w}{\epsilon} \cdot \log\log_n \frac{1}{\epsilon}\right).$$

Corollary15 can be interpreted as an error reduction procedure for PRGs with a slight overhead in the seed and space complexity. We proceed by applying this error reduction to Nisan's PRG from Theorem8.

Corollary16. For any positive integers n, w, any error parameter $1/(4n) > \epsilon > 0$ and any alphabet Σ, there exists an (n, Σ, w, ϵ) WPRG with seed length

$$O\left(\log n \log(nw|\Sigma|) + \log \frac{w}{\epsilon} \cdot \log\log_n \frac{1}{\epsilon}\right)$$

computable in space $O(\log(nw|\Sigma|) + \log\log_n(1/\epsilon) \cdot (\log\log(w/\epsilon))^2)$.

Note that for ϵ which is not tiny the space complexity is dominated by the first term. Specifically, for $\epsilon > 2^{-2^{\log^{1/3} n}}$, $w < 2^{2^{\log^{1/3} n}}$ the space complexity is indeed $O(\log(nw|\Sigma|))$. Had we used INW instead, the space complexity would deteriorate to

$$O\left(\log n \cdot \left[\log\log \frac{nw|\Sigma|}{\epsilon}\right]^2 + \log \frac{w}{\epsilon} \cdot \log\log_n \frac{1}{\epsilon}\right).$$

AppendixA: Proof of Lemma10

We restate Lemma10.

Lemma17. Let $L \in \mathbb{R}^{m \times m}$ be an invertible matrix and $A \in \mathbb{R}^{m \times m}$ such that $\|L^{-1} - A\| \leq \epsilon_0$. For any nonnegative integer k, define

$$R(A, L, k) = \sum_{i=0}^{k} (I - AL)^i A.$$

Then, $\|L^{-1} - R(A, L, k)\| \leq \|L^{-1}\| \cdot \|L\|^{k+1} \cdot \epsilon_0^{k+1}$.

Proof. For any matrix Z, the matrices I and Z commute, and so by a straightforward induction,

$$I - \sum_{i=0}^{k} (I - Z)^i Z = (I - Z)^{k+1}.$$

In particular, for $Z = AL$,

$$I - R(A, L, k) \cdot L = (I - AL)^{k+1}.$$

Thus,

$$\begin{aligned} \|L^{-1} - R(A, L, k)\| &= \|(I - R(A, L, k) \cdot L) \cdot L^{-1}\| \\ &\leq \|L^{-1}\| \cdot \|I - R(A, L, k) \cdot L\| \end{aligned}$$

$$
\begin{aligned}
&\leq \|L^{-1}\| \cdot \|I - AL\|^{k+1} \\
&= \|L^{-1}\| \cdot \|(L^{-1} - A) \cdot L\|^{k+1} \\
&\leq \|L^{-1}\| \cdot \|L\|^{k+1} \cdot \epsilon_0^{k+1}.
\end{aligned}
$$

□

AppendixB: The space complexity of some pseudorandom objects

In this section we show how to achieve the space complexity declared in Theorem8 and Theorem9. For the INW generator we choose a specific implementation with a small space complexity. The constructions are well known, and the variant of INW we use was explored by [12]. We give it here for completeness.

B.1 Nisan's generator

Proof sketch of Theorem8. We are given parameters n, Σ, w, ϵ. We set $X = [A]$ for $A = O(nw\Sigma/\epsilon)$. We let H be a 2-universal family of hash functions over X where $|H| = A^2$ and $h(x)$, for $h \in H$ and $x \in X$, can be computed in space $O(\log\log|X|)$ (see [17,18]). Nisan's generator interprets the seed as $y, h_1, \ldots, h_{\log n}$, where $y \in X$, and $h_1, \ldots, h_{\log n} \in H$. For $j \in [n]$, the j-th symbol in the output of the generator is $h_1^{b_1}(h_2^{b_2} \cdots h_{\log n}^{b_{\log n}}(y))$, where $(b_1, \ldots, b_{\log n}) \in \{0,1\}^{\log n}$ is the binary representation of j, and h^b is either h, if $b = 1$, or the identity function, if $b = 0$. Given $y, h_1, \ldots, h_{\log n}, j = (b_1, \ldots, b_{\log n})$ we can compute the j-th output symbol in the following two alternative ways.

- We can successively compute $h_j^{b_j} \cdots h_{\log n}^{b_{\log n}}(y)$ for $j = \log n, \ldots, 1$, each time keeping the current X-symbol. This takes

$$
O\left(\log\frac{nw|\Sigma|}{\epsilon} + \log\log n + \log\log|X|\right) = O\left(\log\frac{nw|\Sigma|}{\epsilon}\right)
$$

  space.

- Alternatively, we can do the above computation using composition of space bounded reductions, resulting in space complexity

$$
O(\log n \cdot \log\log|X|) = O\left(\log n \cdot \log\log\frac{nw|\Sigma|}{\epsilon}\right).
$$

□

B.2 A high min-entropy extractor

To apply INW, we need a space-efficient seeded extractor with a small entropy loss in the high min-entropy regime. Goldreich and Wigderson [11] gave such a construction utilizing a regular expander $G = (V, E)$ with a small normalized second eigenvalue. For our expander, we choose a Cayley graph over the commutative group $\mathbb{Z}_2^n$ with a generator set $S \subseteq \{0,1\}^n$ that is λ-biased. It is well

known that $\mathrm{Cay}(\mathbb{Z}_2^n, S)$ has normalized second largest eigenvalue at most λ. For the λ-biased set we choose a construction from [2]. Altogether, this unfolds for the following.

- For the λ-biased set S, first pick q to be the first power of two larger than n/λ. The set S is of cardinality q^2. For every $\alpha, \beta \in \mathbb{F}_q$ there is an elements $s_{\alpha,\beta} \in \mathbb{Z}_2^n$ where $(s_{\alpha,\beta})_i = \langle \alpha^i, \beta \rangle$, such that multiplication is in $\mathbb{F}_q$ and the inner product is over $\mathbb{Z}_2$. [2] showed the set is λ-biased.
- We let $G = (V, E)$ with $V = \mathbb{Z}_2^n$ and $(x, y) \in E$ iff $x + y \in S$. G is a λ-expander.

The extractor $\mathrm{GW} : \{0,1\}^n \times [D] \to \{0,1\}^n$ is defined by letting $G(x, i)$ be the i-th neighbour of x in the graph G.

Claim18. Let $0 < \Delta < n$ and set G and GW as above. Then, $\mathrm{GW} : \{0,1\}^n \times [D] \to \{0,1\}^n$ is a $(k = n - \Delta, \epsilon)$ extractor with seed length $d = O(\Delta + \log(n/\epsilon))$ and space complexity $O(\log n \cdot \log(\Delta + \log(n/\epsilon)))$.

Proof. For correctness, note that the expander mixing lemma shows that GW is an $(n - \Delta, \epsilon = O(2^{\Delta/2}\lambda))$ extractor.

Seed length. The seed length of this extractor is $\log |S| = O(\log(n/\lambda)) = O(\log(n2^{\Delta}/\epsilon)) = O(\Delta + \log(n/\epsilon))$.

Space complexity. The space complexity of computing $\mathrm{GW}(x, y)$ given x and y, is the space needed to compute $s_y \in S$ from $y = (\alpha, \beta) \in \mathbb{F}_q^2$, plus the space needed to compute $x + s_y$. The dominating step in computing s_y is computing α^i (for $i \leq n$) which can be done in $O(\log n \log\log q)$ with space composition. Altogether, the space needed is

$$O(\log n \cdot \log\log(n/\lambda)) = O\left(\log n \cdot \log\log \frac{n2^{\Delta}}{\epsilon}\right).$$

We note that Healy and Viola [12] gave an extremely efficient implementation of the above AGHP generator, yielding a better space complexity of $O(\log(n + \log q))$ to compute $\langle \alpha^i, \beta \rangle$. However, in our overall setting of parameters it will make negligible difference. □

We remark that by using expanders with better dependence between D and λ, one can get $d = O(\Delta + \log(1/\epsilon))$, but here we care more about the space complexity, and $\log n$ factors are negligible for us.

B.3 The INW generator

Proof sketch of Theorem9. We consider the INW generator [15] instantiated with extractors (as, e.g., in [21]). We are given parameters n, Σ, w, and $\epsilon = \epsilon_{\mathrm{INW}}$. We set parameters $\Delta = \log w + O(\log(n/\epsilon))$, and d as the seed length for the extractor of Claim18 for length n, error $\epsilon_{\mathrm{Ext}} = \epsilon/n$ and Δ. We let $s = \log |\Sigma| + \log n \cdot 2d$ and we assume $s \leq n$. We let $\ell_i = s - i \cdot \Delta$ for $0 \leq i \leq n$. Given a seed $x \in \{0,1\}^s$ we view the computation of $\mathrm{INW}(x)$ as a full binary tree of depth $\log n$. Nodes in level i of the tree are labeled by strings

of length ℓ_i. The root (at level 0) is labeled by x (of length $\ell_0 = s$). Given any internal node in level $i \in \{0, \ldots, \log n\}$ labeled by some string $z \in \{0,1\}^{\ell_i}$, we write $z = z_1 \circ z_2$ with $z_i \in \{0,1\}^{\ell_{i+1}}$ and $z_2 \in \{0,1\}^d$. The left child of z is labeled with z_1, and the right child of z is labeled with $\mathrm{Ext}_i(z_1, z_2)$, where Ext_i is given by Claim18 for Δ, length ℓ_{i+1} and error ϵ_{Ext} (notice that since $\ell_i < n$, d bits suffice for the seed). $\mathrm{INW}(x)$ is the concatenation of the leaf's labels, from left to right, truncating outputs to $\log|\Sigma|$ bits. Given an index $j \in [n]$, computing $\mathrm{INW}(x)_j \in \Sigma$ can be done by walking down the computation tree, and each time either truncating a string or invoking an extractor. By composition of space bounded reductions the space complexity of the construction is $\log n$ times the space complexity of the worst extractor used. That is, $\log n \cdot \log \ell_0 \cdot \log(\Delta + \log(\ell_0/\epsilon_{\mathrm{Ext}}))$. Plugging-in Δ and ϵ_{Ext}, the space complexity is bounded by

$$\begin{aligned} O\left(\log n \cdot \log \ell_0 \cdot \log\log \frac{nw}{\epsilon}\right) &= O\left(\log n \cdot \log\left(\log|\Sigma| + \log n \log \frac{nw}{\epsilon}\right) \cdot \log\log \frac{nw}{\epsilon}\right) \\ &= O\left(\log n \cdot \left[\log\log \frac{nw|\Sigma|}{\epsilon}\right]^2\right). \end{aligned}$$

□

References

[1] AmirMahdi Ahmadinejad, Jonathan Kelner, Jack Murtagh, John Peebles, Aaron Sidford, and Salil Vadhan. High-precision estimation of random walks in small space. In Proceedings of the 61st Annual IEEE Symposium on Foundations of Computer Science (FOCS 2020), pages 1295–1306. IEEE, 2020.

[2] Noga Alon, Oded Goldreich, Johan Håstad, and René Peralta. Simple constructions of almost k-wise independent random variables. Random Structures & Algorithms, 3(3):289–304, 1992.

[3] Sanjeev Arora and Boaz Barak. Computational Complexity - A Modern Approach. Cambridge University Press, 2009. URL: http://www.cambridge.org/catalogue/catalogue.asp?isbn=9780521424264.

[4] Allan Borodin, Stephen Cook, and Nicholas Pippenger. Parallel computation for well-endowed rings and space-bounded probabilistic machines. Information and Control, 58(1-3):113–136, 1983.

[5] Mark Braverman, Gil Cohen, and Sumegha Garg. Pseudorandom pseudodistributions with near-optimal error for read-once branching programs. SIAM Journal on Computing, 49(5):STOC18–242–STOC18–299, 2020.

[6] Eshan Chattopadhyay and Jyun-Jie Liao. Optimal error pseudodistributions for read-once branching programs. In Proceedings of the 35th Computational Complexity Conference (CCC 2020). Schloss Dagstuhl-Leibniz-Zentrum für Informatik, 2020.

[7] Kuan Cheng and William M. Hoza. Hitting sets give two-sided derandomization of small space. In 35th Computational Complexity Conference (CCC 2020). Schloss Dagstuhl-Leibniz-Zentrum für Informatik, 2020.

[8] Michael B. Cohen, Jonathan Kelner, John Peebles, Richard Peng, Anup B. Rao, Aaron Sidford, and Adrian Vladu. Almost linear-time algorithms for Markov chains and new spectral primitives for directed graphs. In Proceedings of the 49th Annual ACM SIGACT Symposium on Theory of Computing (STOC 2017). ACM, 2017.

[9] Michael B. Cohen, Jonathan Kelner, John Peebles, Richard Peng, Aaron Sidford, and Adrian Vladu. Faster algorithms for computing the stationary distribution, simulating random walks, and more. In Proceedings of the 57th Annual IEEE Symposium on Foundations of Computer Science (FOCS 2016). IEEE, 2016.

[10] Oded Goldreich. Computational complexity: a conceptual perspective. Cambridge University Press, Cambridge, 2008.

[11] Oded Goldreich and Avi Wigderson. Tiny families of functions with random properties: A quality-size trade-off for hashing. Random Structures & Algorithms, 11(4):315–343, 1997.

[12] Alexander Healy and Emanuele Viola. Constant-depth circuits for arithmetic in finite fields of characteristic two. In Annual Symposium on Theoretical Aspects of Computer Science (STACS 2006). Springer, 2006.

[13] William M. Hoza and David Zuckerman. Simple optimal hitting sets for small-success RL. SIAM Journal on Computing, 49(4):811–820, 2020.

[14] Russel Impagliazzo, Valentine Kabanets, and Avi Wigderson. In search of an easy witness: Exponential time vs. probabilistic polynomial time. Journal of Computer and System Sciences, 65(4):672–694, 2002.

[15] Russell Impagliazzo, Noam Nisan, and Avi Wigderson. Pseudorandomness for network algorithms. In Proceedings of the 26th Annual ACM SIGACT Symposium on Theory of Computing (STOC 1994). ACM, 1994.

[16] Valentine Kabanets and Russell Impagliazzo. Derandomizing polynomial identity tests means proving circuit lower bounds. computational complexity, 13(1-2):1–46, 2004.

[17] Noam Nisan. Pseudorandom generators for space-bounded computation. Combinatorica, 12(4):449–461, 1992.

[18] Noam Nisan. RL $\subseteq$ SC. computational complexity, 4(1):1–11, 1994.

[19] Noam Nisan and Avi Wigderson. Hardness vs. randomness. Journal of Computer and System Sciences, 49(2):149–167, 1994.

[20] Edward Pyne and Salil Vadhan. personal communication, February 2021.

[21] Ran Raz and Omer Reingold. On recycling the randomness of states in space bounded computation. In Proceedings of the 31st Annual ACM SIGACT Symposium on Theory of Computing (STOC 1999). ACM, 1999.

[22] Michael E. Saks and Shiyu Zhou. BPH SPACE(S) $\subseteq$ DSPACE(S2/3). Journal of Computer and System Sceinces, 58(2):376–403, 1999.

[23] Walter J. Savitch. Relationships between nondeterministic and deterministic tape complexities. Journal of Computer and System Sciences, 4(2):177–192, April 1970.